# UNDOING TOKENIZATION BIAS: EXACT NEXT-BYTE PREDICTION FROM TOKENIZED LANGUAGE MODELS

**Anonymous authors**
Paper under double-blind review

## ABSTRACT

Modern language models (LMs) operate on tokens rather than bytes, and this discrete compression step has a subtle statistical cost that we call *tokenization bias*: conditioning on the single deterministic encoding of a prompt can make the model's next-character distribution differ from the byte-level distribution it was trained to capture. The effect is most severe when a prompt ends in the middle of a token, as in fill-in-the-middle (FIM) code completion, where even a perfectly trained model can assign zero probability to the correct continuation. We show that tokenization bias is not a limitation of the model but of how it is queried. We formalize valid and invalid encodings for deterministic tokenizers and prove a Byte-Token Representation Lemma that expresses the probability of any byte string as a sum over a small set of *cover encodings*. Building on it, we derive an inference-time algorithm, with a backward search for cover encodings and an incremental update requiring one model call per byte, that turns any tokenized LM into a statistically equivalent byte-level LM without retraining. On Markov chains, the method exactly recovers transition probabilities that the naive token-level model gets wrong. On the HumanEval random-span infilling benchmark with CodeLlama-7b, it raises greedy pass rate in the Suffix-Prefix-Middle setting from 45.0 to 63.9, outperforming token healing and token alignment. Because byte-level predictions share a common output space, the method also enables ensembling of LMs with different vocabularies: averaging the top-2 models improves over the best member on MMLU (63.4 to 65.4) and HumanEval (38.4 to 42.1), although it does not help when members differ widely in capability (GSM8K).

## 1 INTRODUCTION

Nearly every state-of-the-art language model is trained on sequences of *tokens* produced by a subword tokenizer such as byte-pair encoding (BPE) (Gage, 1994; Sennrich et al., 2016), WordPiece (Schuster & Nakajima, 2012; Wu et al., 2016), or SentencePiece (Kudo & Richardson, 2018). Tokenization shortens sequences, which reduces the quadratic cost of self-attention (Vaswani et al., 2017) and has been an essential ingredient of large-scale models (Radford et al., 2019; Brown et al., 2020; Touvron et al., 2023; Jiang et al., 2023a). Yet the text that users write, and the quantity we ultimately care about, is a sequence of bytes. A tokenized LM therefore only indirectly describes the byte-level distribution, through a deterministic and highly non-linear map from bytes to tokens.

This indirection causes well-known failures in practice. When a prompt ends in the middle of what would normally be a single token, the tokenizer is forced to emit an unusual encoding of the prompt, and the model, which has rarely or never seen such an encoding during training, produces poor continuations (Figure 1, left). The problem is pervasive in code completion, where the cursor is frequently placed mid-identifier, and in fill-in-the-middle (FIM) infilling (Bavarian et al., 2022; Fried et al., 2023; Rozière et al., 2023), where the split points are arbitrary characters. Practitioners resort to heuristics such as *token healing* (Dagan et al., 2024), which backs up the prompt by one token and constrains the next generated token, or *token alignment* (Athiwaratkun et al., 2024), which extends this idea to several tokens. A second, related limitation is that models with different vocabularies produce distributions over disjoint output spaces, so their predictions cannot be directly averaged; ensembling or mixing such models requires vocabulary alignment (Xu et al., 2024; Huang et al., 2024; Yu et al., 2024), distillation (Wan et al., 2024), or output re-ranking (Jiang et al., 2023b).

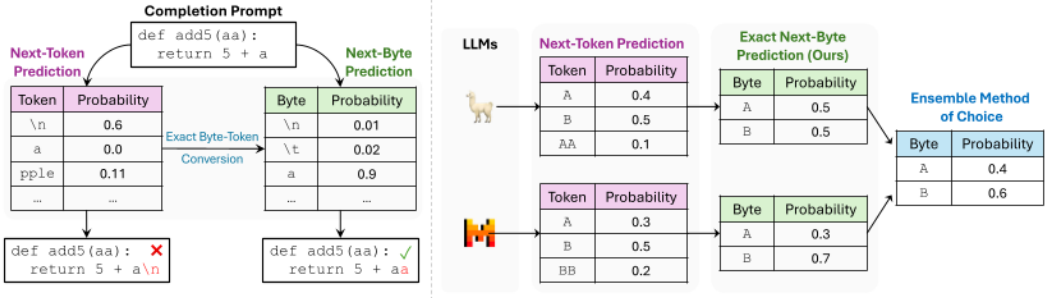



Figure 1: Left: tokenized LMs suffer from tokenization bias when a prompt ends mid-token, as in this code completion example: the correct continuation `a` receives zero probability under next-token prediction. Our method uses the same model but predicts the correct next byte. Right: our method maps the next-token predictions of arbitrary tokenized LMs to statistically equivalent next-byte predictions (Section 3.3), which places all models in a common output space and enables any ensembling strategy, such as averaging or mixture of experts.

In this paper we argue that both problems share a single root cause, which we call *tokenization bias*: the discrepancy between the next-byte distribution implied by conditioning a token-level model on the canonical encoding of the prompt, and the true next-byte distribution of the data. We show that this bias persists even for a *perfect* token-level model, since it arises from conditioning on one particular encoding of the past rather than on the past bytes themselves. The remedy follows from the same observation: if we marginalize over all token sequences that are consistent with the observed bytes, the bias disappears. For deterministic tokenizers, most of these token sequences are *invalid*, i.e., they would never be produced by the tokenizer and therefore have zero probability, and the remaining *cover encodings* can be enumerated efficiently.

Our contributions are as follows.

- **A formal account of tokenization bias.** Using valid and invalid encodings of deterministic tokenizers, we explain why a perfectly trained token-level model can assign zero probability to a correct next byte (Section 3.2).
- **An exact byte-token correspondence.** The Byte-Token Representation Lemma expresses the probability of any byte prefix as a sum over its cover encodings (Section 3.3).
- **An efficient, training-free next-byte sampler.** A backward search for cover encodings and an incremental update with one model call per byte turn any tokenized LM into a byte-level LM (Sections 3.4–3.5).
- **Empirical evidence.** The method recovers Markov-chain transition probabilities, closes most of the PSM–SPM gap in FIM infilling (SPM greedy pass rate $45.0 \rightarrow 63.9$), outperforms token healing and token alignment, and enables ensembles of models with different tokenizers. We also document where it does not help (Section 4).

## 2 Related Work

**Tokenization and its theory.** Subword tokenizers such as BPE (Gage, 1994; Sennrich et al., 2016), WordPiece (Schuster & Nakajima, 2012; Wu et al., 2016), and the unigram model of SentencePiece (Kudo, 2018; Kudo & Richardson, 2018) are the default front-end of modern LMs; see Mielke et al. (2021) for a survey. A growing literature studies how tokenization affects modeling quality, e.g., through compression (Zouhar et al., 2023; Bostrom & Durrett, 2020), fairness across languages (Petrov et al., 2023), and its role in enabling transformers to learn sequential structure (Rajaraman et al., 2024). Our focus is complementary: we take a tokenizer and a trained model as given and characterize the exact relationship between the induced token-level and the original byte-level distribution.

**Marginalizing over tokenizations.** Because many token sequences decode to the same string, the probability of a string is in principle a sum over its tokenizations. Cao & Rimell (2021) and Chirkova et al. (2023) estimate this marginal likelihood with importance sampling and find that the canonical tokenization usually dominates for complete strings; subword regularization (Kudo, 2018; Provilkov et al., 2020) exposes models to non-canonical tokenizations during training; Geh et al. (2024) study the signal contained in non-canonical tokenizations at inference; and Pimentel & Meister (2024) show how to correctly compute word probabilities under tokenizers whose tokens carry leading whitespace. Our setting differs in two ways. First, we condition on a byte *prefix* that may end mid-token, where the canonical encoding is precisely the problematic one. Second, we exploit the determinism of the tokenizer: invalid encodings have zero probability, which reduces the exponentially many tokenizations to a small, efficiently enumerable set of cover encodings and makes the computation exact rather than approximate.

**Byte-level and tokenizer-free models.** An alternative is to avoid tokenization altogether by modeling characters or bytes directly (Al-Rfou et al., 2019; Xue et al., 2022; Clark et al., 2022; Tay et al., 2022; Yu et al., 2023; Wang et al., 2024). Such models avoid tokenization bias but must be trained from scratch on longer sequences; our approach instead converts an existing tokenized LM at inference time. Zero-shot tokenizer transfer (Minixhofer et al., 2024) also changes the tokenizer of a trained model, but relies on a learned hypernetwork and yields an approximation rather than an exactly equivalent distribution.

**Prompt boundaries and code infilling.** FIM training (Bavarian et al., 2022; Fried et al., 2023; Rozière et al., 2023) enables models to infill code given a prefix and a suffix, usually in the Prefix-Suffix-Middle (PSM) or Suffix-Prefix-Middle (SPM) format. Bavarian et al. (2022) introduced the HumanEval random-span infilling benchmark built on HumanEval (Chen et al., 2021), in which spans start and end at arbitrary characters. To handle prompts ending mid-token, token healing (Dagan et al., 2024) removes the last prompt token and constrains the first generated token to start with the removed characters, while token alignment (Athiwaratkun et al., 2024) backtracks several tokens and constrains generation with character matching. Both are heuristics that do not compute the correct conditional distribution in general; we compare against both in Section 4.3.

**Ensembling language models.** Ensembles improve accuracy and calibration in many settings (Dietterich, 2000; Lakshminarayanan et al., 2017), and output aggregation by majority voting is common for LMs (Wang et al., 2023). For LMs with different vocabularies, probability-level ensembling is hindered by the mismatch of output spaces. Existing approaches rank and fuse complete outputs (Jiang et al., 2023b), distill several models into one (Wan et al., 2024), learn or construct mappings between vocabularies (Xu et al., 2024; Huang et al., 2024), or align tokens heuristically (Yu et al., 2024). Mapping each model to exact next-byte probabilities provides a common, lossless output space in which any combination rule, including simple averaging or a mixture of experts (Shazeer et al., 2017), can be applied at every generation step.

# 3 METHODOLOGY

We first formalize the relationship between bytes and tokens (Section 3.1) and define tokenization bias (Section 3.2). We then derive an exact representation of byte probabilities in terms of token probabilities (Section 3.3), an efficient search procedure (Section 3.4), and the resulting next-byte sampler and ensembling scheme (Sections 3.5–3.6). Proofs are deferred to Appendix A.

## 3.1 SETUP AND NOTATION

Let $\mathcal{A}$ be a finite alphabet of bytes (in practice, the 256 values of UTF-8 bytes, together with an end-of-document symbol). We write $x_1^n = x_1 x_2 \cdots x_n \in \mathcal{A}^n$ for a byte string and $x_i^j$ for its substring between positions $i$ and $j$. A tokenizer consists of a finite vocabulary $\mathcal{V} \subset \mathcal{A}^+$ of tokens, each a non-empty byte string, together with an encoding function $\mathrm{encode} : \mathcal{A}^* \to \mathcal{V}^*$ and a decoding function $\mathrm{decode} : \mathcal{V}^* \to \mathcal{A}^*$ that concatenates the bytes of the tokens. We assume that $\mathcal{V}$ contains every single byte, so that every string can be encoded, and that $\mathrm{decode}(\mathrm{encode}(x_1^n)) = x_1^n$. For token sequences we write $t_1^k = t_1 \cdots t_k \in \mathcal{V}^k$, $\vec{t} \oplus \tau$ for appending a token $\tau$, and $|\cdot|$ for the length in bytes of a token or decoded string. We focus on *deterministic* tokenizers, namely BPE (Sennrich

et al., 2016), where $\mathrm{encode}$ applies a fixed ordered list of merge rules, and maximum-prefix encoding (MPE), the greedy longest-match rule used by WordPiece (Wu et al., 2016).

Documents $X$ are drawn from a byte-level distribution. We use $P(x_1^n)$ to denote the probability that a document begins with $x_1^n$, and for a token sequence $\vec{t} = t_1^k$ we use $P(t_1^k)$ for the probability that $\mathrm{encode}(X)$ begins with $t_1^k$. A tokenized LM is trained to model the conditionals $P(t_{k+1} \mid t_1^k)$ of this induced token process, from which $P(t_1^k) = \prod_{i=1}^{k} P(t_i \mid t_1^{i-1})$. For our theoretical statements we consider an idealized LM that matches these conditionals exactly; in experiments we simply plug in the trained model.

### 3.2 Valid encodings and tokenization bias

A deterministic tokenizer never produces most token sequences. For example, with vocabulary $\{\texttt{A}, \texttt{B}, \texttt{AA}\}$ and MPE, the string AA is always encoded as the single token AA, never as A·A.

**Definition 1** (Valid and invalid encodings). *A token sequence $t_1^k$ is* valid *if* $\mathrm{encode}(\mathrm{decode}(t_1^k)) = t_1^k$*, and* invalid *otherwise.*

**Assumption 1** (Prefix validity). *For every document $x$ and every $k$, the prefix $t_1^k$ of $\mathrm{encode}(x)$ is valid.*

Assumption 1 states that cutting a canonical encoding at a token boundary leaves a canonical encoding of the cut string. It holds for MPE (Appendix A) and for BPE, since in BPE no merge is ever applied across a final token boundary, so the merges applied to the left part are unaffected by the right part (we discuss this argument and its edge cases in Appendix A). Two consequences follow immediately: if $t_1^k$ is valid then so is each prefix $t_1^j$, $j < k$; and since the event $\{\mathrm{encode}(X) \text{ begins with } t_1^k\}$ is empty for invalid $t_1^k$, we have

$$P(t_1^k) = 0 \quad \text{for every invalid } t_1^k. \tag{1}$$

Eq. (1) is the source of tokenization bias. Consider the naive way of obtaining a next-byte distribution from a tokenized LM: encode the prompt and sum the probabilities of next tokens that start with a given byte $a$,

$$\tilde{P}(x_{n+1} = a \mid x_1^n) := \sum_{\tau \in \mathcal{V}:\, \tau_1 = a} P\big(\tau \;\big|\; \mathrm{encode}(x_1^n)\big). \tag{2}$$

**Definition 2** (Tokenization bias). *A token-level model exhibits* tokenization bias *at $x_1^n$ if $\tilde{P}(x_{n+1} \mid x_1^n) \neq P(x_{n+1} \mid x_1^n)$ for some $x_{n+1} \in \mathcal{A}$.*

The naive estimate in Eq. (2) silently conditions on the event that a token boundary falls exactly at position $n$. This event is correlated with what follows: the tokenizer places a boundary after $x_1^n$ only if the continuation does not form a longer token with the last bytes of the prompt. Hence, even the exact token process can be biased.

**Example 1** (First-order Markov chain). *Let $\mathcal{A} = \{\texttt{A}, \texttt{B}\}$ and let the data follow a Markov chain with $P(\texttt{B} \mid \texttt{A}) = \alpha$, $P(\texttt{A} \mid \texttt{A}) = 1 - \alpha$, $P(\texttt{A} \mid \texttt{B}) = \beta$ and $P(\texttt{B} \mid \texttt{B}) = 1 - \beta$. With vocabulary $\{\texttt{A}, \texttt{B}, \texttt{AA}\}$ and either MPE or BPE, the token A can only be followed by a token starting with B, because otherwise the two A's would have been merged into AA. Thus for any prompt whose encoding ends with token A, the naive estimate gives $\tilde{P}(\texttt{A} \mid x_1^n) = 0$, whereas the true probability is $1-\alpha$ (Figure 2).*

This is exactly what happens when a prompt ends mid-token in practice (Figure 1): the prompt `return 5 + a` is encoded with a final token a, while in the training data the identifier aa would be a single token, so the model has learned that a is never followed by a token starting with a.

### 3.3 The Byte-Token Representation Lemma

To remove the bias, we must compute $P(x_1^n)$ directly, without assuming a token boundary at position $n$. The key object is the set of token sequences whose *last* token is the one that covers the final byte $x_n$.

**Definition 3** (Cover encodings). *The set of cover encodings of $x_1^n$, denoted $\mathrm{cover}(x_1^n)$, contains all valid token sequences $t_1^k$ ($k \geq 1$) such that $\mathrm{decode}(t_1^{k-1}) = x_1^i$ for some $i < n$ and the last token $t_k$ begins with $x_{i+1}^n$.*

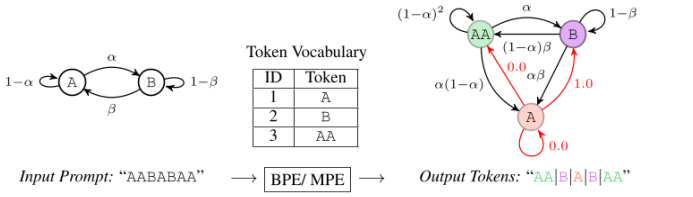



Figure 2: Tokenization bias on a first-order Markov chain (Example 1). Given the context token A, the model never samples a next token starting with A, although the correct probability is $1 - \alpha$. This occurs when prompts end with a token that is part of a longer token, as is common in FIM tasks.

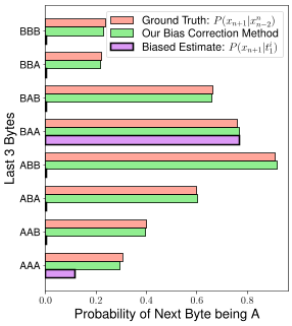



Figure 3: Third-order Markov chain: our conversion (Section 3.5) recovers $P(x_{n+1}|x_{n-2}^n)$.

In words, a cover encoding tokenizes the prompt up to some position $i < n$ and then places a single token that spans the remaining bytes $x_{i+1}^n$ and may extend beyond position $n$ (Figure 4, left). Note that $\text{decode}(t_1^k)$ has $x_1^n$ as a prefix but may be longer.

**Lemma 1** (Byte-Token Representation)**.** *Under Assumption 1, for every byte string $x_1^n$ with $n \geq 1$,*

$$P(x_1^n) = \sum_{\vec{t} \in \text{cover}(x_1^n)} P(\vec{t}). \tag{3}$$

The proof (Appendix A) observes that the event "the document begins with $x_1^n$" is the disjoint union, over $\vec{t} \in \text{cover}(x_1^n)$, of the events "the encoding of the document begins with $\vec{t}$": each document beginning with $x_1^n$ has exactly one shortest prefix of its encoding whose decoding reaches position $n$, and that prefix is valid by Assumption 1. Each term $P(\vec{t})$ is a product of conditionals of the tokenized LM. Lemma 1 immediately gives an unbiased next-byte distribution,

$$P(x_{n+1} \mid x_1^n) = \frac{\sum_{\vec{t} \in \text{cover}(x_1^{n+1})} P(\vec{t})}{\sum_{\vec{t} \in \text{cover}(x_1^n)} P(\vec{t})}. \tag{4}$$

Restricting the sum to valid sequences is what makes Eq. (3) exact (invalid sequences have zero probability under the data, even if a trained model assigns them mass), and it reduces the exponentially many segmentations of $x_1^n$ to a small set, as we show next.

### 3.4 Efficient Search for Cover Encodings

Two structural facts make $\text{cover}(x_1^n)$ easy to enumerate. First, the prefix $t_1^{k-1}$ of a cover encoding is valid and decodes to exactly $x_1^i$; since a valid sequence satisfies $t_1^{k-1} = \text{encode}(\text{decode}(t_1^{k-1}))$, it must equal $\text{encode}(x_1^i)$. Hence a cover encoding is determined by the start position $i + 1$ of its last token and by the last token itself. Second, the last token must contain the suffix $x_{i+1}^n$ as a prefix, so only $n - i \leq L$ needs to be considered, where $L = \max_{\tau \in \mathcal{V}} |\tau|$ is the maximum token length.

Algorithm 1 therefore searches *backward* from the end of the string. At iteration $j = 1, 2, \ldots$, it encodes the prefix $x_1^{n-j}$, collects every token that begins with the last $j$ bytes $x_{n-j+1}^n$, and keeps the combined sequence $\text{encode}(x_1^{n-j}) \oplus \tau$ only if it is valid. The probability of each accepted sequence is $P(\text{encode}(x_1^{n-j}))\, P(\tau \mid \text{encode}(x_1^{n-j}))$, which needs one forward pass of the LM per iteration. The search terminates as soon as no token in $\mathcal{V}$ has length at least $j$ (in the example of Figure 4, no token has more than three characters). Hence the procedure requires at most $L$ model calls, independent of the context length $n$, and never traverses the full history.

### 3.5 Exact Next-Byte Prediction

Running Algorithm 1 at every generation step would cost up to $L$ model calls per byte. Instead, we update the cover set incrementally. The following proposition, proven in Appendix A, shows that

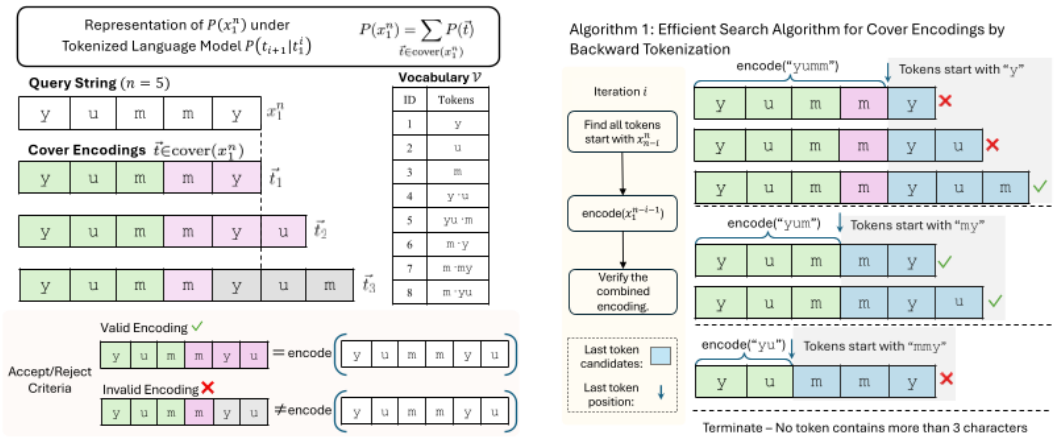



Figure 4: Left: representation of $P(x_1^n)$ with a tokenized LM for the query string `yummy`, showing its cover encodings and examples of a valid and an invalid encoding. Right: illustration of Algorithm 1 (cover encoding search). Green ticks and red crosses denote valid and invalid encodings (Definition 1). The search terminates once no token is long enough to span the remaining suffix. We use BPE in this example.

**Algorithm 1** Cover encoding search by backward tokenization

**Require:** byte string $x_1^n$; tokenizer $(\mathcal{V}, \text{encode}, \text{decode})$; tokenized LM $P(t_{k+1} \mid t_1^k)$
**Ensure:** $\text{cover}(x_1^n)$ with probabilities $P(\vec{t})$

$\mathcal{C} \leftarrow \emptyset$
**for** $j = 1, \ldots, \min(n, L)$ **do**
  $\vec{p} \leftarrow \text{encode}(x_1^{n-j})$ {empty sequence if $j = n$}
  compute $P(\vec{p})$ and $P(\cdot \mid \vec{p})$ with one forward pass of the LM
  **for all** $\tau \in \mathcal{V}$ such that $\tau$ begins with $x_{n-j+1}^n$ **do**
    **if** $\text{encode}(\text{decode}(\vec{p} \oplus \tau)) = \vec{p} \oplus \tau$ **then**
      $\mathcal{C} \leftarrow \mathcal{C} \cup \{(\vec{p} \oplus \tau,\ P(\vec{p})\, P(\tau \mid \vec{p}))\}$ {accept valid encoding}
    **end if**
  **end for**
**end for**
**return** $\mathcal{C}$

cover encodings of $x_1^{n+1}$ arise either from cover encodings of $x_1^n$ whose last token already extends past position $n$, or from appending a new token to the canonical encoding $\text{encode}(x_1^n)$.

**Proposition 1** (Incremental cover update)**.** *Under Assumption 1, for any* $x_1^{n+1}$, $\text{cover}(x_1^{n+1})$ *is the disjoint union of*

$$\mathcal{C}_{n+1}^{\text{old}} = \{\vec{t} \in \text{cover}(x_1^n) \ :\ |\text{decode}(\vec{t})| \geq n+1 \textit{ and } \text{decode}(\vec{t})_{n+1} = x_{n+1}\}, \tag{5}$$

$$\mathcal{C}_{n+1}^{\text{new}} = \{\text{encode}(x_1^n) \oplus \tau \ :\ \tau \in \mathcal{V},\ \tau_1 = x_{n+1},\ \text{encode}(x_1^n) \oplus \tau \textit{ is valid}\}. \tag{6}$$

*Moreover,* $\text{cover}(x_1^n) = \{\text{encode}(x_1^n)\} \cup \{\vec{t} \in \text{cover}(x_1^n) : |\text{decode}(\vec{t})| > n\}$.

Combining Proposition 1 with Lemma 1, the unnormalized probability of each candidate next byte $a \in \mathcal{A}$ is

$$P(x_1^n a) = \underbrace{\sum_{\vec{t} \in \mathcal{C}_{n+1}^{\text{old}}(a)} P(\vec{t})}_{\text{(i) previous cover encodings}} + \underbrace{P\big(\text{encode}(x_1^n)\big) \sum_{\tau:\ \text{encode}(x_1^n) \oplus \tau \in \mathcal{C}_{n+1}^{\text{new}}(a)} P\big(\tau \,\big|\, \text{encode}(x_1^n)\big)}_{\text{(ii) new cover encodings}}, \tag{7}$$

where $\mathcal{C}_{n+1}^{\text{old}}(a)$ and $\mathcal{C}_{n+1}^{\text{new}}(a)$ denote the sets in Eqs. (5)–(6) with $x_{n+1} = a$, i.e., previous cover encodings whose $(n+1)$-th byte is $a$, and valid one-token extensions of $\text{encode}(x_1^n)$ whose new

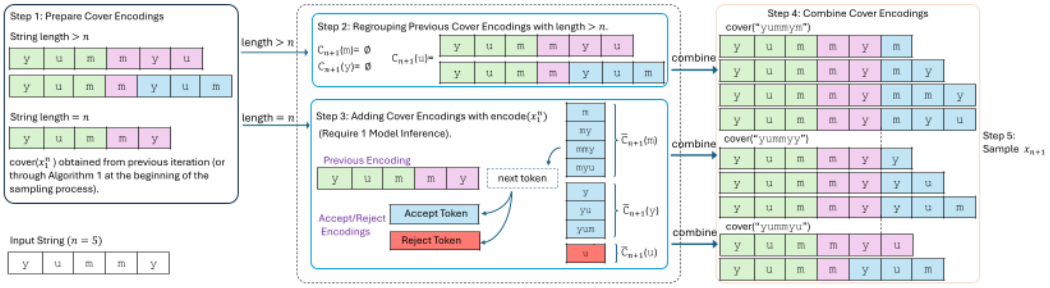



Figure 5: Illustration of sampling $x_{n+1}$ from a tokenized LM $P(t_{k+1}|t_1^k)$, continuing the example of Figure 4. Probabilities $P(\vec{t})$ are omitted for simplicity. Steps 1–5 are described in Section 3.5; only Step 3 requires a model call.

token starts with $a$. The next-byte distribution is $P(x_{n+1} = a \mid x_1^n) = P(x_1^n a) / \sum_{a' \in \mathcal{A}} P(x_1^n a')$. Compared with the naive estimate in Eq. (2), Eq. (7) adds term (i), the mass of encodings in which a single token spans positions $n$ and $n+1$, which the naive estimate ignores entirely, and restricts term (ii) to valid continuations, whereas the naive estimate also counts invalid ones. In Example 1, term (i) contains the encoding ending with AA, which restores the correct probability $1 - \alpha$.

The resulting procedure is illustrated in Figure 5 and summarized as follows. **Step 1:** obtain $\mathrm{cover}(x_1^n)$, either from the previous step or, at the start of generation, from Algorithm 1. **Step 2:** regroup the cover encodings whose decoding is longer than $n$ according to their byte at position $n+1$. **Step 3:** run one forward pass on $\mathrm{encode}(x_1^n)$ and add the valid one-token extensions, grouped by their first byte. **Step 4:** combine both groups to form $\mathrm{cover}(x_1^n a)$ and $P(x_1^n a)$ for every $a$. **Step 5:** sample (or choose greedily) $x_{n+1}$ from the normalized distribution and retain $\mathrm{cover}(x_1^{n+1})$ for the next step. In the idealized setting, this produces exactly $P(x_{n+1} \mid x_1^n)$; with a trained model, renormalization over valid continuations discards the mass that the model places on invalid encodings.

**Cost.** Generation needs one model call per output *byte* (plus at most $L$ calls to initialize a new prompt), so it is slower than token-level generation by roughly the average number of bytes per token. The cover set is bounded by the number of tokens that can span the current position. Implementation details (log-space bookkeeping, key-value caching, control tokens, beam search) are in Appendix B.

### 3.6 Ensembling models with different vocabularies

Since Eq. (7) yields a distribution over the shared byte alphabet $\mathcal{A}$, models $P^{(1)}, \ldots, P^{(M)}$ with arbitrary, mutually incompatible tokenizers can be combined at every step. Each model maintains its own cover set for the common byte history $x_1^n$, and the ensemble distribution is

$$P_{\mathrm{ens}}(x_{n+1} \mid x_1^n) = \sum_{m=1}^{M} w_m \, P^{(m)}(x_{n+1} \mid x_1^n), \qquad w_m \geq 0, \; \textstyle\sum_m w_m = 1. \tag{8}$$

We use uniform weights $w_m = 1/M$ in all experiments, but the byte space admits any combination rule, such as a product of experts or a learned, context-dependent gating in the spirit of mixture-of-experts (Shazeer et al., 2017). The sampled byte is appended to the shared history and every member updates its cover set independently.

## 4 Experiments

We study tokenization bias with known ground truth (Section 4.2), mid-token prompts in code infilling (Section 4.3), cross-tokenizer ensembles (Section 4.4), and beam search (Section 4.5).

### 4.1 SETUP

**Models.** For synthetic experiments we use first- and third-order Markov chains over a binary alphabet and a GPT-2 architecture (Radford et al., 2019) with 6 hidden layers trained on tokenized sequences sampled from the chain. For natural language we use three general-purpose open-weight LMs with different tokenizers: Llama2-7B (Touvron et al., 2023), Yi-1.5-6B (Young et al., 2024), and Mistral-7B-v0.3 (Jiang et al., 2023a). For code we use CodeLlama-7b and CodeLlama-13b (Rozière et al., 2023) (denoted CodeLlama2-7b and CodeLlama2-13b in Table 2) and Yi-Coder-1.5B. All models are transformer LMs, used without any weight updates.

**Tasks and metrics.** For infilling we use the HumanEval random-span benchmark (Bavarian et al., 2022), built on HumanEval (Chen et al., 2021), and report pass rates for greedy decoding and pass@10. We evaluate both standard FIM prompt formats, PSM and SPM. In PSM the middle is generated after a sentinel token, i.e., from a token boundary; in SPM the prefix immediately precedes the generated middle, so when the span starts mid-token the prompt ends mid-token. For knowledge and reasoning we use MMLU (Hendrycks et al., 2021), PIQA (Bisk et al., 2020), Natural Questions (Kwiatkowski et al., 2019), TriviaQA (Joshi et al., 2017), and GSM8K (Cobbe et al., 2021), and for code generation HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021), all reported as accuracy or pass@1. For each benchmark we compare the same model queried at the token level (standard decoding or token-level likelihoods) and at the byte level (our method), using the same prompts.

**Baselines.** For infilling, we compare standard token-level prediction, token-level prediction with token healing (Dagan et al., 2024), and token-level prediction with token alignment (Athiwaratkun et al., 2024). The two heuristics target prompts that end mid-token and are applied in SPM mode. For ensembling, the byte-level ensemble averages the next-byte distributions (Eq. (8), uniform weights) of the two best individual models on each benchmark; we compare it against each member and, for multiple-choice tasks, against majority voting over the same top-2 models.

### 4.2 TOKENIZATION BIAS ON MARKOV CHAINS

Markov chains provide exact ground-truth conditionals. In the first-order chain of Example 1, the token-level model conditioned on a context ending with token `A` assigns probability $0.0$ to the next byte being `A`, while the true probability is $1 - \alpha$ (Figure 2). This is not a training artifact but the correct behavior of the induced token process (Eq. (1)). For the third-order chain, Figure 3 compares, for every 3-byte context, the true probability that the next byte is `A`, the naive estimate $\tilde{P}(x_{n+1} \mid t_1^k)$ of the trained tokenized model, and our byte-level estimate from the same model. The naive estimate is severely biased (near zero for most contexts), whereas our conversion closely recovers $P(x_{n+1} \mid x_{n-2}^n)$ for all eight contexts: the model contains the byte-level information, and the bias comes only from how it is queried.

### 4.3 FILL-IN-THE-MIDDLE CODE COMPLETION

Figure 6 reports pass rates of CodeLlama-7b on HumanEval random-span infilling.

**Mid-token prompts severely hurt token-level decoding.** Switching from PSM to SPM reduces the greedy pass rate of the token-level model from 64.9 to 45.0, and pass@10 from 84.0 to 66.5. The model and the information content of the prompt are identical in both formats; the difference is that SPM prompts often end mid-token, producing out-of-distribution encodings and, as in Example 1, zero probability on the correct continuation.

**Byte-level prediction removes the gap.** With our method, SPM greedy pass rate rises from 45.0 to 63.9 (an improvement of 18.9 points) and pass@10 from 66.5 to 84.3, which is on par with PSM (65.7 greedy, 84.1 pass@10). The temperature sweep in Figure 6 (right) shows that the byte-level PSM and SPM curves nearly coincide at all temperatures, whereas the token-level SPM curve stays far below PSM.

**Comparison with heuristics.** Token healing and token alignment recover most of the SPM degradation (greedy 62.7 and 63.0; pass@10 83.8 and 82.9), but remain below the byte-level method (63.9 and 84.3). The heuristics repair the last one or few prompt tokens but do not compute the correct

| Predictions | PSM Greedy | PSM @10 | SPM Greedy | SPM @10 |
|---|---|---|---|---|
| Token-level | 64.9 | 84.0 | 45.0 | 66.5 |
| + healing | \| | \| | 62.7 | 83.8 |
| + alignment | \| | \| | 63.0 | 82.9 |
| Byte-level | **65.7** | **84.1** | **63.9** | **84.3** |

@10 here is the max over the temperature sweep on the right

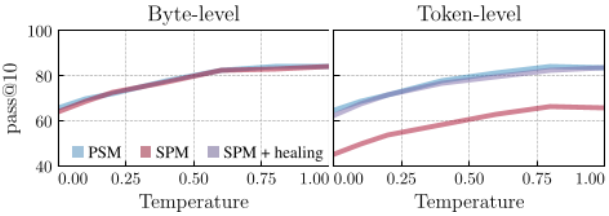



Figure 6: Pass rates on the HumanEval random-span infilling benchmark (Bavarian et al., 2022) with CodeLlama-7b. Left: greedy and pass@10 for the PSM and SPM prompt formats, with token healing (Dagan et al., 2024) and token alignment (Athiwaratkun et al., 2024) as baselines (pass@10 is the maximum over the temperature sweep). Right: pass@10 as a function of sampling temperature for byte-level (ours) and token-level predictions. The SPM format forces predictions from the middle of a token, which byte-level predictions handle successfully.

Table 1: General-purpose LMs on knowledge and reasoning benchmarks: token-level vs. byte-level (ours) predictions of single models, top-2 majority voting, and the byte-level top-2 ensemble (ours). N/A: not applicable or not evaluated. Bold marks the best entry per benchmark.

| | MMLU | | PIQA | | NQ | | TriviaQA | | GSM8K | |
|---|---|---|---|---|---|---|---|---|---|---|
| Model | Token | Byte | Token | Byte | Token | Byte | Token | Byte | Token | Byte |
| Llama2-7B | 45.4 | 45.7 | 78.1 | 78.2 | 25.0 | 23.4 | 58.4 | 58.1 | 15.1 | 12.5 |
| Yi-1.5-6B | 63.4 | 63.4 | 78.5 | 78.5 | 22.8 | 22.7 | 53.7 | 53.4 | 61.3 | **61.5** |
| Mistral-7B-v0.3 | 62.1 | 62.1 | 80.1 | 80.3 | 28.5 | 28.8 | 63.6 | 63.5 | 39.0 | 39.0 |
| Voting (top-2) | N/A | 62.1 | N/A | 80.1 | N/A | N/A | N/A | N/A | N/A | N/A |
| Top-2 ensemble (ours) | N/A | **65.4** | N/A | **80.7** | N/A | **30.0** | N/A | **64.2** | N/A | 55.8 |

conditional (Appendix D); our method is exact under the idealized model and needs no choice of how many tokens to back up.

**Gains even without mid-token prompts.** In PSM mode, where generation starts at a token boundary, byte-level greedy decoding still slightly outperforms token-level greedy decoding (65.7 vs. 64.9), and pass@10 is essentially unchanged (84.1 vs. 84.0). A plausible explanation is that the byte-level distribution aggregates probability across all tokens sharing a prefix, so greedy byte decoding is less sensitive to the split of probability mass among similar tokens; we have not isolated this effect.

**Qualitative analysis.** Inspecting individual failures, we found cases in which the prompt ends in the middle of an identifier, e.g., partial forms of `delimeter`, `collatz`, and `palindrome`. The token-level model fails to complete the identifier, whose completing token is incompatible with the canonical encoding of the truncated word, while the byte-level method completes it correctly.

### 4.4 Byte-level ensembles of models with different tokenizers

Tables 1 and 2 report token-level and byte-level scores of each model and the byte-level top-2 ensemble (token-level ensembles are impossible because the members' output spaces differ).

**Ensembles improve over their members when members are comparable.** On MMLU, the byte-level ensemble of Yi-1.5-6B and Mistral-7B-v0.3 reaches 65.4, compared with 63.4 for the best member, while top-2 voting obtains 62.1. The ensemble also exceeds the best single model on PIQA (80.7 vs. 80.3), NQ (30.0 vs. 28.8), and TriviaQA (64.2 vs. 63.6). On HumanEval, the ensemble of Yi-Coder-1.5B and CodeLlama2-13b achieves 42.1, which is 3.7 points above the best individual score (38.4, Yi-Coder-1.5B at the token level) and 5.6 points above the best byte-level single model. On MBPP the ensemble matches the best byte-level member (53.6) and improves over the best token-level score (52.8) by 0.8 points.

Table 2: Code LMs on HumanEval and MBPP (pass@1): token-level vs. byte-level (ours) predictions and the byte-level top-2 ensemble (ours). Bold: best per benchmark.

| | HumanEval@1 | | MBPP@1 | |
|---|---|---|---|---|
| Model | Token | Byte | Token | Byte |
| CodeLlama2-7b | 32.3 | 28.7 | 40.6 | 42.4 |
| CodeLlama2-13b | 35.8 | 33.5 | 47.6 | 47.4 |
| Yi-Coder-1.5B | 38.4 | 36.5 | 52.8 | **53.6** |
| Top-2 ensemble (ours) | N/A | **42.1** | N/A | **53.6** |

Table 3: Beam width for byte-level prediction on HumanEval random-span infilling (CodeLlama-7b, pass rate). Width 1 is greedy.

| Beam width | PSM | SPM |
|---|---|---|
| 1 | 65.7 | 63.9 |
| 2 | 66.8 | 65.9 |
| 4 | **67.7** | **66.8** |

**Ensembles fail when members are too divergent.** On GSM8K, Yi-1.5-6B outperforms Mistral-7B-v0.3 by more than 20 points (61.5 vs. 39.0 at the byte level), and the ensemble (55.8) is worse than the stronger member. With uniform weights, the weaker model contributes half of the mass at every byte of a long reasoning chain, so errors can compound; members of comparable capability or non-uniform weights appear necessary.

**Byte-level vs. token-level single models.** For likelihood-based multiple-choice tasks (MMLU, PIQA), byte-level and token-level accuracies are nearly identical (differences of at most 0.3 points), as expected when the scored continuations begin at a token boundary and the canonical encoding dominates. For generation tasks the picture is mixed. Byte-level decoding is within 0.3 points on most entries and improves some (e.g., MBPP for CodeLlama2-7b, 40.6 to 42.4), but it reduces HumanEval pass@1 for all three code models (by 1.9 to 3.6 points) and lowers Llama2-7B on NQ (25.0 to 23.4) and GSM8K (15.1 to 12.5). Two factors could contribute: a trained model places mass on invalid encodings, which our method discards (exactness holds only for the idealized model), and greedy search over bytes is more myopic than greedy search over tokens. The beam-search results below are consistent with the latter, but we have not separated the two. Byte-level conversion is thus most valuable where tokenization bias is actually present (mid-token prompts, cross-vocabulary ensembles), not as a universal replacement for token-level decoding.

### 4.5 Ablation: beam search over bytes

Table 3 varies the beam width of byte-level decoding on the infilling task. Increasing the beam width from 1 to 4 consistently improves pass rates, by 2.0 points in PSM (65.7 to 67.7) and by 2.9 points in SPM (63.9 to 66.8). With beam width 4, the byte-level SPM result exceeds the token-level PSM greedy result (64.9), showing that once tokenization bias is removed, the SPM format is no longer a handicap. The steady improvement with beam width suggests that part of the remaining gap is due to the myopia of byte-by-byte greedy search.

## 5 Conclusion

We identified tokenization bias, which affects even perfectly trained tokenized LMs, and showed that byte-level probabilities can be computed exactly from token-level ones by summing over cover encodings. The resulting training-free algorithm recovers Markov-chain transition probabilities, removes most of the SPM degradation in FIM infilling (greedy pass rate 45.0 to 63.9) while outperforming token healing and token alignment, and enables ensembles of models with different tokenizers that improve over the best member on MMLU, PIQA, NQ, TriviaQA, and HumanEval.

**Limitations.** The exactness guarantee assumes a deterministic tokenizer with the prefix-validity property and an idealized model; trained models place mass on invalid encodings, and the effect of discarding this mass is not fully understood. Byte-level decoding costs one model call per byte, which makes generation slower than token-level decoding. Byte-level greedy decoding of single models underperforms token-level decoding on several generation benchmarks (notably HumanEval), and uniform ensembles degrade when members differ widely in capability (GSM8K). Our results come from single evaluation runs without confidence intervals, and the infilling study uses one model. **Future work** includes stochastic tokenizers, reducing the per-byte cost (e.g., applying the conversion only near prompt boundaries), and learned ensemble weights in byte space.

## Reproducibility Statement

The method requires no training and is fully specified by Definitions 1–3, Lemma 1, Proposition 1, Algorithm 1, and Eqs. (7)–(8). Proofs are given in Appendix A. All evaluated models and benchmarks are publicly available, the evaluation protocol is described in Section 4.1, and implementation details are given in Appendix B.

## References


Rami Al-Rfou, Dokook Choe, Noah Constant, Mandy Guo, and Llion Jones. Character-level language modeling with deeper self-attention. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pp. 3159–3166, 2019.

Ben Athiwaratkun, Shiqi Wang, Mingyue Shang, Yuchen Tian, Zijian Wang, Sujan Kumar Gonugondla, Sanjay Krishna Gouda, Rob Kwiatkowski, Ramesh Nallapati, and Bing Xiang. Token alignment via character matching for subword completion. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Mohammad Bavarian, Heewoo Jun, Nikolas Tezak, John Schulman, Christine McLeavey, Jerry Tworek, and Mark Chen. Efficient training of language models to fill in the middle. *arXiv preprint arXiv:2207.14255*, 2022.

Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. PIQA: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pp. 7432–7439, 2020.

Kaj Bostrom and Greg Durrett. Byte pair encoding is suboptimal for language model pretraining. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 4617–4624, 2020.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pp. 1877–1901, 2020.

Kris Cao and Laura Rimell. You should evaluate your language model on marginal likelihood over tokenisations. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 2104–2114, 2021.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Nadezhda Chirkova, Germán Kruszewski, Jos Rozen, and Marc Dymetman. Should you marginalize over possible tokenizations? In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pp. 1–12, 2023.

Jonathan H. Clark, Dan Garrette, Iulia Turc, and John Wieting. CANINE: Pre-training an efficient tokenization-free encoder for language representation. *Transactions of the Association for Computational Linguistics*, 10:73–91, 2022.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Gautier Dagan, Gabriel Synnaeve, and Baptiste Rozière. Getting the most out of your tokenizer for pre-training and domain adaptation. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.

Thomas G. Dietterich. Ensemble methods in machine learning. In *International Workshop on Multiple Classifier Systems*, pp. 1–15, 2000.

Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen-tau Yih, Luke Zettlemoyer, and Mike Lewis. InCoder: A generative model for code infilling and synthesis. In *International Conference on Learning Representations*, 2023.

Philip Gage. A new algorithm for data compression. *The C Users Journal*, 12(2):23–38, 1994.

Renato Lui Geh, Honghua Zhang, Kareem Ahmed, Benjie Wang, and Guy Van den Broeck. Where is the signal in tokenization space? *arXiv preprint arXiv:2408.08541*, 2024.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021.

Yichong Huang, Xiaocheng Feng, Baohang Li, Yang Xiang, Hui Wang, Bing Qin, and Ting Liu. Enabling ensemble learning for heterogeneous large language models with deep parallel collaboration. *arXiv preprint arXiv:2404.12715*, 2024.

Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. Mistral 7B. *arXiv preprint arXiv:2310.06825*, 2023a.

Dongfu Jiang, Xiang Ren, and Bill Yuchen Lin. LLM-Blender: Ensembling large language models with pairwise ranking and generative fusion. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 14165–14178, 2023b.

Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1601–1611, 2017.

Taku Kudo. Subword regularization: Improving neural network translation models with multiple subword candidates. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 66–75, 2018.

Taku Kudo and John Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 66–71, 2018.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 2019.

Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Sabrina J. Mielke, Zaid Alyafeai, Elizabeth Salesky, Colin Raffel, Manan Dey, Matthias Gallé, Arun Raja, Chenglei Si, Wilson Y. Lee, Benoît Sagot, et al. Between words and characters: A brief history of open-vocabulary modeling and tokenization in NLP. *arXiv preprint arXiv:2112.10508*, 2021.

Benjamin Minixhofer, Edoardo Maria Ponti, and Ivan Vulić. Zero-shot tokenizer transfer. *arXiv preprint arXiv:2405.07883*, 2024.

Aleksandar Petrov, Emanuele La Malfa, Philip H. S. Torr, and Adel Bibi. Language model tokenizers introduce unfairness between languages. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Tiago Pimentel and Clara Meister. How to compute the probability of a word. *arXiv preprint arXiv:2406.14561*, 2024.

Ivan Provilkov, Dmitrii Emelianenko, and Elena Voita. BPE-dropout: Simple and effective subword regularization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 1882–1892, 2020.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.

Nived Rajaraman, Jiantao Jiao, and Kannan Ramchandran. Toward a theory of tokenization in LLMs. *arXiv preprint arXiv:2404.08335*, 2024.

Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. Code Llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

Mike Schuster and Kaisuke Nakajima. Japanese and Korean voice search. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 5149–5152, 2012.

Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1715–1725, 2016.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017.

Yi Tay, Vinh Q. Tran, Sebastian Ruder, Jai Gupta, Hyung Won Chung, Dara Bahri, Zhen Qin, Simon Baumgartner, Cong Yu, and Donald Metzler. Charformer: Fast character transformers via gradient-based subword tokenization. In *International Conference on Learning Representations*, 2022.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Fanqi Wan, Xinting Huang, Deng Cai, Xiaojun Quan, Wei Bi, and Shuming Shi. Knowledge fusion of large language models. In *International Conference on Learning Representations*, 2024.

Junxiong Wang, Tushaar Gangavarapu, Jing Nathan Yan, and Alexander M. Rush. MambaByte: Token-free selective state space model. *arXiv preprint arXiv:2401.13660*, 2024.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023.

Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V. Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. Google's neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016.

Yangyifan Xu, Jinliang Lu, and Jiajun Zhang. Bridging the gap between different vocabularies for LLM ensemble. *arXiv preprint arXiv:2404.09492*, 2024.

Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 10:291–306, 2022.

Alex Young, Bei Chen, Chao Li, Chengen Huang, Ge Zhang, Guanwei Zhang, Heng Li, Jiangcheng Zhu, Jianqun Chen, Jing Chang, et al. Yi: Open foundation models by 01.AI. *arXiv preprint arXiv:2403.04652*, 2024.

Lili Yu, Dániel Simig, Colin Flaherty, Armen Aghajanyan, Luke Zettlemoyer, and Mike Lewis. MEGABYTE: Predicting million-byte sequences with multiscale transformers. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Yao-Ching Yu, Chun-Chih Kuo, Ziqi Ye, Yu-Cheng Chang, and Yueh-Se Li. Breaking the ceiling of the LLM community by treating token generation as a classification for ensembling. *arXiv preprint arXiv:2406.12585*, 2024.

Vilém Zouhar, Clara Meister, Juan Gastaldi, Li Du, Mrinmaya Sachan, and Ryan Cotterell. Tokenization and the noiseless channel. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 5184–5207, 2023.

# A PROOFS

Throughout, a document $x$ is a finite byte string, $\mathrm{encode}(x) = t_1^K$ is its canonical encoding, and for a token sequence $\vec{t}$ we write $E(\vec{t})$ for the event that $\mathrm{encode}(X)$ begins with $\vec{t}$, so that $P(\vec{t}) = \Pr[E(\vec{t})]$.

**Prefix validity for MPE.** Let encode be maximum-prefix encoding and let $t_1^K = \mathrm{encode}(x)$. Fix $k \le K$ and let $y = \mathrm{decode}(t_1^k)$. MPE selects as $t_1$ the longest token that is a prefix of $x$. Since $t_1$ is a prefix of $y$ and every token that is a prefix of $y$ is also a prefix of $x$ (as $y$ is a prefix of $x$), $t_1$ is also the longest token that is a prefix of $y$. Applying the same argument to the remaining string $x$ with $t_1$ removed and proceeding inductively shows that MPE applied to $y$ produces $t_1, \ldots, t_k$ and then stops, so $\mathrm{encode}(y) = t_1^k$, i.e., $t_1^k$ is valid.

**Prefix validity for BPE (argument and caveats).** BPE starts from single bytes and repeatedly merges adjacent pairs according to a fixed priority list. In $\mathrm{encode}(x)$, the position between $t_k$ and $t_{k+1}$ is never merged across. The merges performed inside $y = \mathrm{decode}(t_1^k)$ therefore form a valid merge sequence for $y$ alone, and the pairs available inside $y$ at every stage are the same in both runs, except for the pair straddling the boundary, which by assumption is never merged. Hence BPE applied to $y$ performs the same merges and yields $t_1^k$. A formal treatment must account for implementation details such as tie-breaking among overlapping occurrences of the same pair (e.g., runs of identical bytes); for standard left-to-right implementations the argument goes through, and in any case the validity of every candidate encoding is checked explicitly in Algorithm 1 and Eq. (7).

**Proof of Eq. (1).** If $E(\vec{t})$ occurs, then $\vec{t}$ is a prefix of $\mathrm{encode}(X)$ and is valid by Assumption 1. Hence $E(\vec{t}) = \emptyset$ for invalid $\vec{t}$. □

**Proof of Lemma 1.** Let $F$ denote the event that $X$ begins with $x_1^n$. We show that $F$ is the disjoint union of $E(\vec{t})$ over $\vec{t} \in \mathrm{cover}(x_1^n)$.

*($\supseteq$)* Let $\vec{t} = t_1^k \in \mathrm{cover}(x_1^n)$ and suppose $E(\vec{t})$ occurs. Then $X$ begins with $\mathrm{decode}(t_1^k)$, which begins with $x_1^n$ by Definition 3, so $F$ occurs.

*($\subseteq$)* Suppose $F$ occurs and let $\mathrm{encode}(X) = t_1^K$. Since $\mathrm{decode}(t_1^K) = X$ has length at least $n$, there is a smallest index $k$ such that $|\,\mathrm{decode}(t_1^k)| \ge n$. By minimality, $\mathrm{decode}(t_1^{k-1}) = x_1^i$ with $i < n$, and since $\mathrm{decode}(t_1^k)$ is a prefix of $X$ of length at least $n$, $t_k$ begins with $x_{i+1}^n$. Moreover, $t_1^k$ is valid by Assumption 1. Hence $t_1^k \in \mathrm{cover}(x_1^n)$ and $E(t_1^k)$ occurs.

*(Disjointness)* Suppose $E(\vec{t})$ and $E(\vec{s})$ both occur for distinct $\vec{t} = t_1^k, \vec{s} = s_1^l \in \mathrm{cover}(x_1^n)$. Both are prefixes of $\mathrm{encode}(X)$, so one is a proper prefix of the other; say $k < l$ and $s_1^k = t_1^k$. Then $|\,\mathrm{decode}(s_1^{l-1})| \ge |\,\mathrm{decode}(t_1^k)| \ge n$, contradicting the requirement $|\,\mathrm{decode}(s_1^{l-1})| < n$ in Definition 3.

Therefore $P(x_1^n) = \Pr[F] = \sum_{\vec{t} \in \mathrm{cover}(x_1^n)} \Pr[E(\vec{t})] = \sum_{\vec{t} \in \mathrm{cover}(x_1^n)} P(\vec{t})$. □

**Proof of Proposition 1.** Let $\vec{t} = t_1^k \in \mathrm{cover}(x_1^{n+1})$ with $\mathrm{decode}(t_1^{k-1}) = x_1^i$, $i \le n$. If $i < n$, then $t_k$ begins with $x_{i+1}^{n+1}$, hence with $x_{i+1}^n$, so $\vec{t} \in \mathrm{cover}(x_1^n)$, $|\,\mathrm{decode}(\vec{t})| \ge n+1$ and $\mathrm{decode}(\vec{t})_{n+1} = x_{n+1}$; thus $\vec{t} \in \mathcal{C}_{n+1}^{\mathrm{old}}$. If $i = n$, then $t_1^{k-1}$ is a prefix of the valid sequence $\vec{t}$ and hence valid (apply Assumption 1 to the document $\mathrm{decode}(\vec{t})$), and it decodes to $x_1^n$, so $t_1^{k-1} = \mathrm{encode}(\mathrm{decode}(t_1^{k-1})) = \mathrm{encode}(x_1^n)$; since $t_k$ begins with $x_{n+1}$, $\vec{t} \in \mathcal{C}_{n+1}^{\mathrm{new}}$. Conversely,

every element of $\mathcal{C}^{\text{old}}_{n+1}$ is valid, its last token starts at some $i+1 \le n$ and begins with $x_{i+1}^{n+1}$, so it belongs to $\text{cover}(x_1^{n+1})$; every element of $\mathcal{C}^{\text{new}}_{n+1}$ is valid, its prefix decodes to $x_1^n$, and its last token begins with $x_{n+1}$, so it also belongs to $\text{cover}(x_1^{n+1})$. The two sets are disjoint because their last tokens start at positions $\le n$ and $n+1$, respectively.

For the second statement, an element $\vec{t} \in \text{cover}(x_1^n)$ with $|\,\text{decode}(\vec{t})| = n$ is valid and decodes to $x_1^n$, hence equals $\text{encode}(x_1^n)$. Conversely, $\text{encode}(x_1^n)$ is valid, non-empty for $n \ge 1$, and its last token starts before position $n+1$ and ends at position $n$, so it belongs to $\text{cover}(x_1^n)$. □

**Correctness of the sampler.** By Lemma 1 applied to $x_1^n a$ and Proposition 1, $P(x_1^n a) = \sum_{\vec{t} \in \mathcal{C}^{\text{old}}_{n+1}} P(\vec{t}) + \sum_{\vec{t} \in \mathcal{C}^{\text{new}}_{n+1}} P(\vec{t})$ with $x_{n+1} = a$, and $P(\text{encode}(x_1^n) \oplus \tau) = P(\text{encode}(x_1^n))\, P(\tau \mid \text{encode}(x_1^n))$, which is Eq. (7). Since the alphabet includes an end-of-document symbol, $\sum_a P(x_1^n a) = P(x_1^n)$, so normalizing over $a$ yields $P(x_{n+1} \mid x_1^n)$ exactly for the idealized model. □

## B Implementation details

All probabilities are kept in log space, and we store the cover-encoding weights normalized by $P(x_1^n)$, which avoids underflow on long sequences. Because $\text{encode}(x_1^{n+1})$ usually shares a long prefix with $\text{encode}(x_1^n)$, the forward pass in Step 3 can reuse the key-value cache and only recompute the re-tokenized suffix. The validity check $\text{encode}(\text{decode}(\vec{t})) = \vec{t}$ is a tokenizer call that does not involve the model. Prompts that contain special control tokens (e.g., FIM sentinel tokens) are handled by treating the control-token prefix as fixed context: all probabilities are conditioned on it, and the byte-level computation is applied to the text that follows the last control token. Greedy decoding chooses the most probable next byte, and beam search (Section 4.5) keeps the $B$ byte prefixes with the highest probability, each with its own cover set. The output of the procedure is a byte stream; multi-byte UTF-8 characters are produced byte by byte.

## C Worked example of the cover encoding search

We trace Algorithm 1 on the example of Figure 4, with query string $x_1^5 = \texttt{yummy}$ and the vocabulary $\{\texttt{y}, \texttt{u}, \texttt{m}, \texttt{yu}, \texttt{yum}, \texttt{my}, \texttt{mmy}, \texttt{myu}\}$ (BPE). At $j = 1$, the algorithm encodes $\texttt{yumm}$ and considers all tokens beginning with $\texttt{y}$; appending $\texttt{y}$ or $\texttt{yu}$ to $\text{encode}(\texttt{yumm}) = \texttt{yum} \cdot \texttt{m}$ yields invalid sequences (re-encoding would merge $\texttt{m}$ and $\texttt{y}$), while appending $\texttt{yum}$ yields the valid sequence $\texttt{yum} \cdot \texttt{m} \cdot \texttt{yum}$. At $j = 2$, it encodes $\texttt{yum}$ and considers the tokens $\texttt{my}$ and $\texttt{myu}$, both of which give valid sequences. At $j = 3$, it encodes $\texttt{yu}$ and considers the token $\texttt{mmy}$; the sequence $\texttt{yu} \cdot \texttt{mmy}$ is invalid, since BPE encodes $\texttt{yummy}$ as $\texttt{yum} \cdot \texttt{my}$. Since no token contains more than three characters, the search terminates. The three accepted sequences $\texttt{yum} \cdot \texttt{m} \cdot \texttt{yum}$, $\texttt{yum} \cdot \texttt{my}$, and $\texttt{yum} \cdot \texttt{myu}$ constitute $\text{cover}(\texttt{yummy})$, and $P(\texttt{yummy})$ is the sum of their probabilities by Lemma 1. Figure 5 continues this example by computing the next-byte distribution: cover encodings whose last token extends past position 5 are regrouped by their sixth byte (Step 2), and one forward pass on $\text{encode}(\texttt{yummy})$ provides the new valid one-token extensions (Step 3).

## D Relationship to token healing

Token healing (Dagan et al., 2024) removes the last token $t_k$ of $\text{encode}(x_1^n)$ and samples the next token from $P(\cdot \mid t_1^{k-1})$ restricted to tokens that begin with $\text{decode}(t_k)$. In the notation of Section 3.4, this corresponds to keeping only the cover encodings whose last token starts at the beginning of $t_k$, without checking validity and without the remaining cover encodings. It therefore coincides with our method only when the omitted cover encodings carry negligible mass and the model puts no mass on invalid continuations. Token alignment (Athiwaratkun et al., 2024) backs up several tokens and is thus closer to the full backward search, but it still conditions on a single choice of how far to back up rather than summing over all cover encodings. This difference is consistent with the gap between the heuristics and the byte-level method in Figure 6.